JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class RecursiveTask<V>

java.lang.Object
  java.util.concurrent.ForkJoinTask<V>
    java.util.concurrent.RecursiveTask<V>

Type Parameters:

V - the type of the result of the task

All Implemented Interfaces:

Serializable, Future<V>

```
public abstract class RecursiveTask<V>
extends ForkJoinTask<V>
```

A recursive result-bearing ForkJoinTask.

For example, here is a task-based program for computing Factorials:

```
import java.util.concurrent.RecursiveTask;
import java.math.BigInteger;
public class Factorial {
  static class FactorialTask extends RecursiveTask<BigInteger> {
    private final int from, to;
    FactorialTask(int from, int to) { this.from = from; this.to = to; }
    protected BigInteger compute() {
      int range = to - from;
      if (range == 0) {                       // base case
        return BigInteger.valueOf(from);
      } else if (range == 1) {                // too small to parallelize
        return BigInteger.valueOf(from).multiply(BigInteger.valueOf(to));
      } else {                                // split in half
        int mid = from + range / 2;
        FactorialTask leftTask = new FactorialTask(from, mid);
        leftTask.fork();         // perform about half the work locally
        return new FactorialTask(mid + 1, to).compute()
               .multiply(leftTask.join());
      }
    }
  }
  static BigInteger factorial(int n) { // uses ForkJoinPool.commonPool()
    return (n <= 1) ? BigInteger.ONE : new FactorialTask(1, n).invoke();
  }
  public static void main(String[] args) {
    System.out.println(factorial(Integer.parseInt(args[0])));
```

```
  }
}
```

Since:

1.7

See Also:

Serialized Form

Nested Class Summary

Nested classes/interfaces declared in interface Future
Future.State

Constructor Summary

Constructors

Constructor	Description
RecursiveTask()	Constructor for subclasses to call.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
protected abstract V	compute()	The main computation performed by this task.
protected final boolean	exec()	Implements execution conventions for RecursiveTask.
final V	getRawResult()	Returns the result that would be returned by ForkJoinTask.join(), even if this task completed abnormally, or null if this task is not known to have been completed.
	setRawResult(V value)	

protected final void	Forces the given value to be returned as a result.

Methods declared in class ForkJoinTask

adapt, adapt, adapt, adaptInterruptible, adaptInterruptible, adaptInterruptible, cancel, compareAndSetForkJoinTaskTag, complete, completeExceptionally, exceptionNow, fork, get, get, getException, getForkJoinTaskTag, getPool, getQueuedTaskCount, getSurplusQueuedTaskCount, helpQuiesce, inForkJoinPool, invoke, invokeAll, invokeAll, invokeAll, isCancelled, isCompletedAbnormally, isCompletedNormally, isDone, join, peekNextLocalTask, pollNextLocalTask, pollSubmission, pollTask, quietlyComplete, quietlyInvoke, quietlyJoin, quietlyJoin, quietlyJoinUninterruptibly, reinitialize, resultNow, setForkJoinTaskTag, state, tryUnfork

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

RecursiveTask

```
public RecursiveTask()
```

Constructor for subclasses to call.

Method Details

compute

```
protected abstract V compute()
```

The main computation performed by this task.

Returns:

the result of the computation

getRawResult

```
public final V getRawResult()
```

Description copied from class: ForkJoinTask

Returns the result that would be returned by `ForkJoinTask.join()`, even if this task completed abnormally, or `null` if this task is not known to have been completed. This method is designed to aid debugging, as well as to support extensions. Its use in any other context is discouraged.

Specified by:

`getRawResult` in class `ForkJoinTask<V>`

Returns:

the result, or `null` if not completed

setRawResult

```
protected final void setRawResult(V value)
```

Description copied from class: `ForkJoinTask`

Forces the given value to be returned as a result. This method is designed to support extensions, and should not in general be called otherwise.

Specified by:

`setRawResult` in class `ForkJoinTask<V>`

Parameters:

`value` - the value

exec

```
protected final boolean exec()
```

Implements execution conventions for RecursiveTask.

Specified by:

`exec` in class `ForkJoinTask<V>`

Returns:

`true` if this task is known to have completed normally

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.